Software Engineering Project 1

Software Requirements Specification

Introduction

This project aims to build an all-in-one web portal that makes everyday campus life easier. Instead of relying on paper forms and visiting physical offices, students and staff can handle almost everything online. The platform is broken down into five main areas: a secure login system, a Student Affairs section for class schedules and official documents, and a Facilities module for things like digital gate passes, cab-sharing, and medical info. It also includes a detailed Hostel Management system to take care of leave requests, mess changes, and laundry, plus a Complaint tracker so students can report issues and actually see them get fixed. Ultimately, the goal is to cut down on paperwork and save time for everyone.

Module 1: Student Affairs & Information Management

1. Stakeholder Identification

Stakeholder	Role	Key Interactions
Student	Primary User	Views profile data, customised Timetable and Exam Schedule, submits update requests for change in their data along with proofs, pays for ID card renewal/replacement, and tracks application status of all kinds of requests.
Admin (Student Affairs)	Decision Maker	Reviews and approves/rejects data updates, ID card requests, and certificate applications.
Finance Section	Financial Verifier	Validates TransactionIDs for ID card renewals or replacements to verify whether transaction has been completed or not before processing.

2. Data Entities & Attributes

2.1 Student Core

Attribute	Data Type	Description
Roll Number	String	Unique identifier for each student
Student Name	String	Full name of the student
Department	String	Department of the student
Programme	String	Degree program (e.g., B.Tech, M.Tech)
Home Address	Text	Permanent residential address
College Address	Text	Hostel/college residence address
Blood Group	String	Blood group of the student
Contact Number	String	Primary contact number
Email ID	String	Official email address
Emergency Contact	String	Emergency contact number
Date of Joining	Date	Date when student joined
Course ID	String	Identifier for enrolled course

2.2 Academic Details (Managed by Admin)

Attribute	Data Type	Description
Course Name	String	Name of the course
Professor Name	String	Instructor assigned to the course
Class Slot	String	Scheduled class timing
Exam Type	String	Type of exam (e.g., Mid-term, End-term)
Exam Day	Date	Date of examination
Exam Room Number	String	Exam hall/room allocation

2.3 Admin & Request Block

Attribute	Data Type	Description
Request ID	Integer	Unique identifier for each request
Request Date	Date	Date when request was submitted
Request Type	String	Type of request (e.g., Bonafide, Renewal, Field Update)
Request Status	String	Current status (Pending, Approved, Rejected, Dispatched)
Admin Role	String	Role of admin handling the request

2.4 Data Change & Logs

Attribute	Data Type	Description
Old Value	Text	Previous value of the field before update
New Value	Text	Updated value of the field
Field Name (Implicit)	String	Field being modified (optional but assumed in context)

2.5 Digital Assets (Vault & Files)

Attribute	Data Type	Description
File Name	String	Name of the uploaded file
File Content	Binary Data	Actual stored file (documents, proofs, etc.)

3. Functional Requirements

FR1: The system must dynamically filter the Timetable and Exam Schedule based on the student's active Roll Number and display the customized Timetable and Exam Schedule for every student.

FR2: The system must synchronize with the database such that whenever the admin adds, modifies, or deletes a course for a student, the change is immediately reflected in the personal dashboard.

FR3: Student submits field-level change request (e.g., address, contact); system stores old and new values with proof; admin approves or rejects with remarks; student is notified on status change.

FR4: System shall provide an option for ID card renewal/replacement. In case of renewal, store proof (FIR) and forward to admin for approval. In case of replacement, submit old and new details along with proof documents.

FR5: System shall provide a fee gateway through which students can pay the fine.

FR6: System shall maintain a status tracker for requests (e.g., when request is submitted, approved, and ID card dispatched).

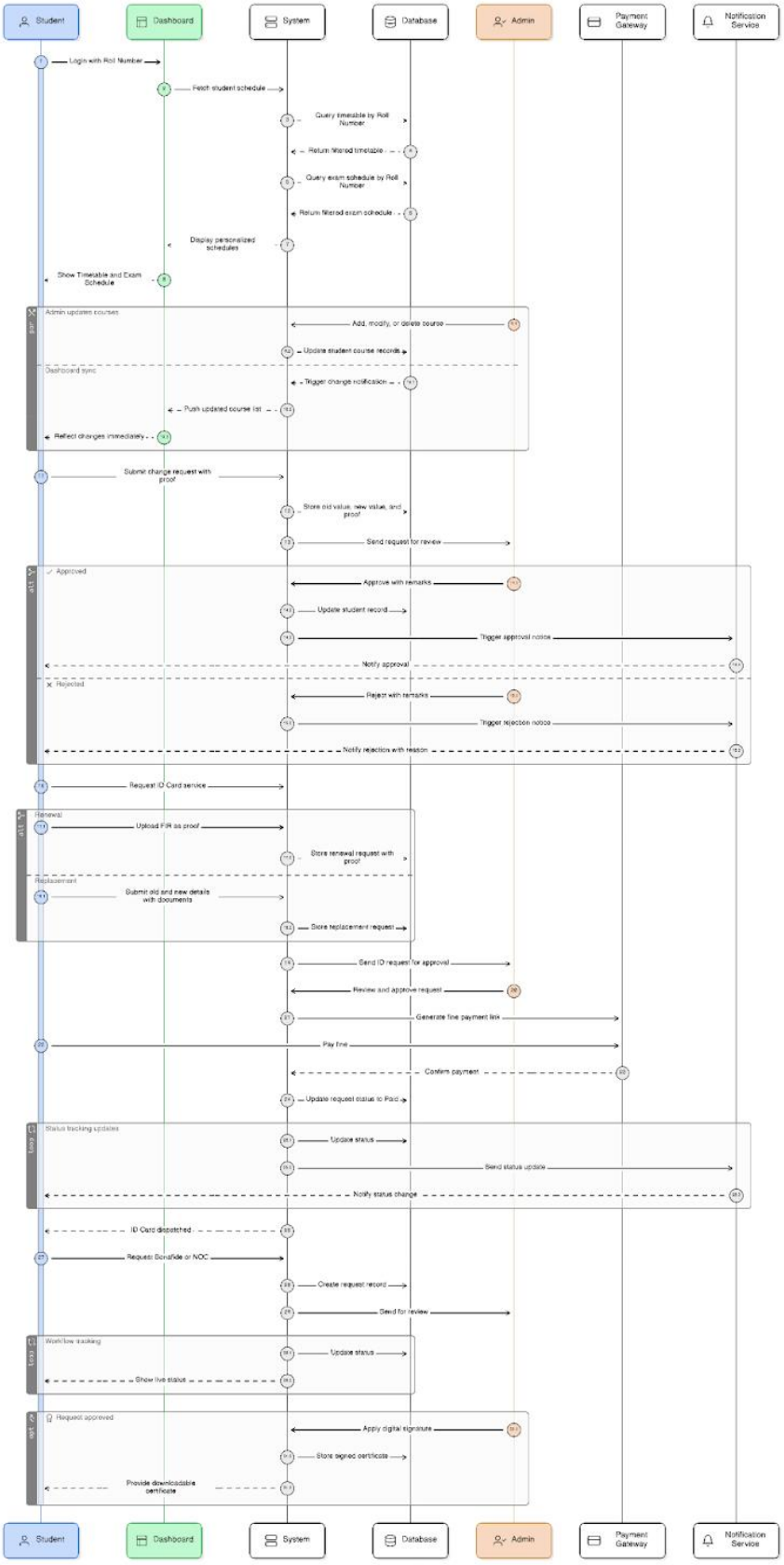
FR7: Request Bonafide Certificate or NOC (Passport/Internship) with workflow tracking: Submitted, Under Review, Completed; admin applies digital signature on approval.

FR8: System shall provide storage such that students can download certificates generated upon completion of NOC request.

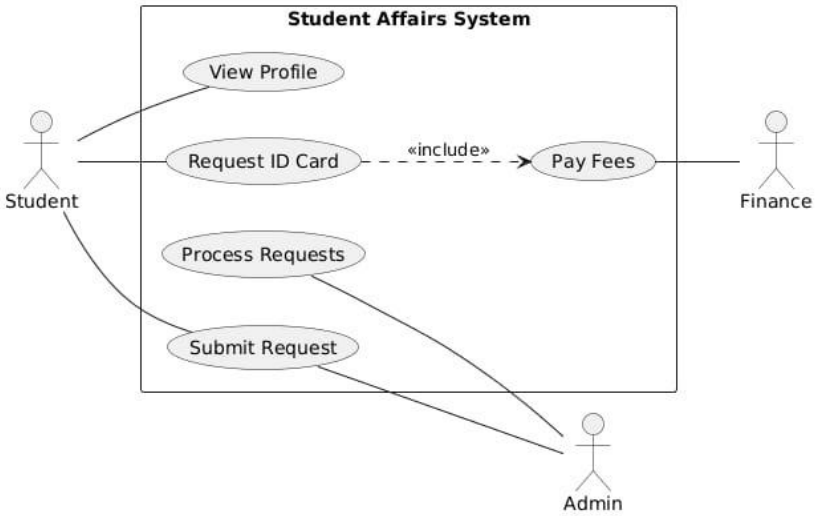
FR9: System shall provide a notification service so that students are notified upon completion of their request.

4. UML Diagrams

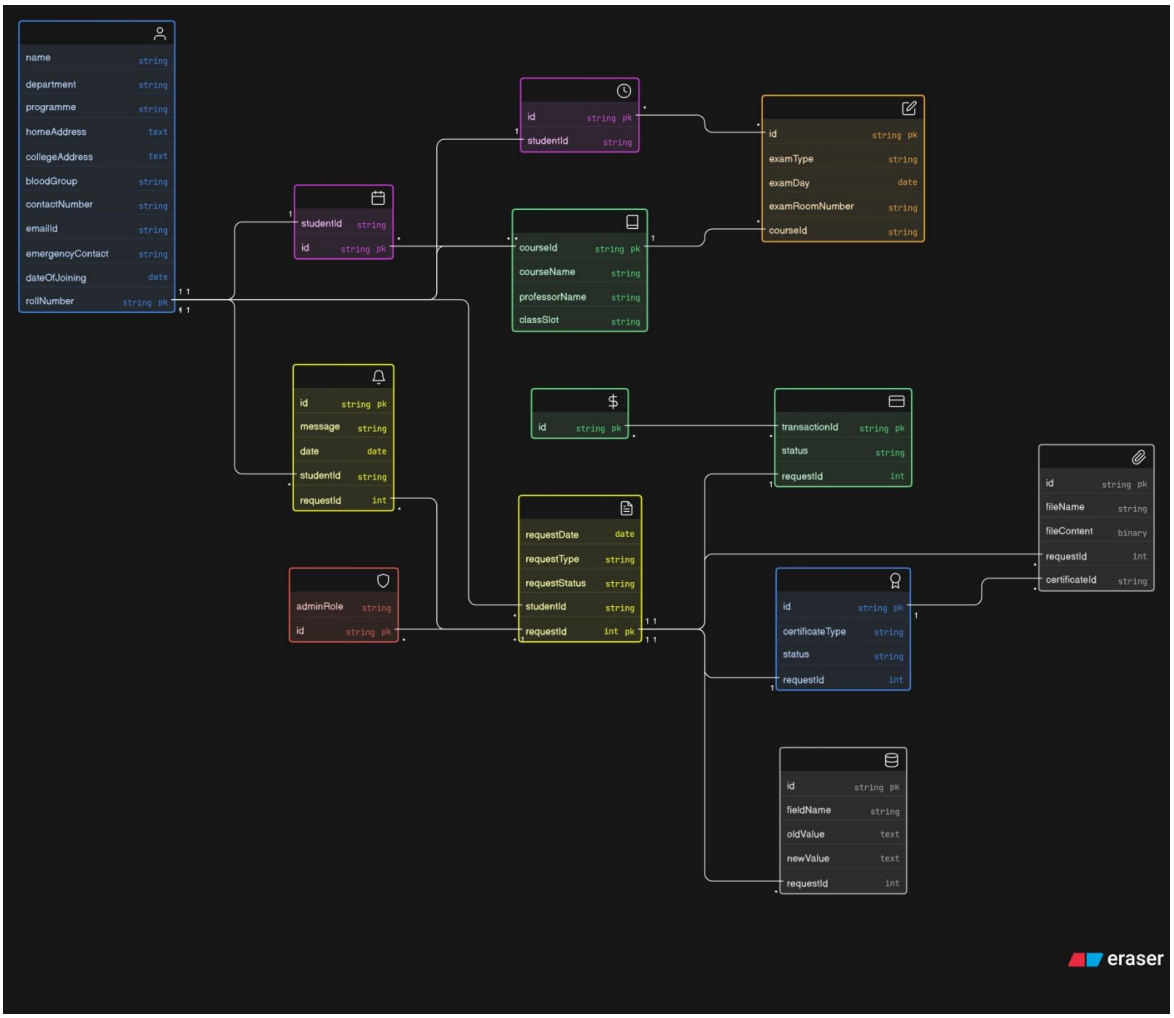
4.1 Sequence Diagram


Student
Dashboard
System
Database
Admin
Payment Gateway
Notification Service
Login with Roll Number
Fetch student schedule
Query timetable by Roll Number
Return filtered timetable
Query exam schedule by Roll Number
Return filtered exam schedule
Display personalized schedules
Show Timetable and Exam Schedule
par
Admin updates courses
Add, modify, or delete course
Update student course records
Dashboard sync
Trigger change notification
Push updated course list
Reflect changes immediately
Submit change request with proof
Store old value, new value, and proof
Send request for review
alt
Approved
Approve with remarks
Update student record
Trigger approval notice
Notify approval
Rejected
Reject with remarks
Trigger rejection notice
Notify rejection with reason
Request ID Card service
alt
Renewal
Upload FIR as proof
Store renewal request with proof
Replacement
Submit old and new details with documents
Store replacement request
Send ID request for approval
Review and approve request
Generate fine payment link
Pay fine
Confirm payment
Update request status to Paid
loop
Status tracking updates
Update status
Send status update
Notify status change
ID Card dispatched
Request Bonafide or NOC
Create request record
Send for review
loop
Workflow tracking
Update status
Show live status
opt
Request approved
Apply digital signature
Store signed certificate
Provide downloadable certificate

4.2 Use Case Diagram



4.3 Class Diagram



Module 2: Facilities, Welfare and Transport

1. Stakeholder Identification

The student is the sole user of this module. All other roles (admin, medical staff, etc.) are backend stakeholders whose interactions are handled through developer-side APIs, admin dashboards, or database coordination — not through this app.

2. Data Entities & Attributes

Users

Attribute	Type	Notes
UserID	INT (PK, AUTO_INCREMENT)	Primary identifier
RollNumber	VARCHAR(20)	Unique student roll number
Email	VARCHAR(100)	Institute email
FullName	VARCHAR(100)	
PhoneNumber	VARCHAR(15)	
HostelName	VARCHAR(50)	
RoomNumber	VARCHAR(10)	

GateLogs

Attribute	Type	Notes
LogID	INT (PK, AUTO_INCREMENT)	
UserID	INT	FK -> Users.UserID
GateLocation	VARCHAR(50)	e.g., 'Main Gate', 'North Gate'
Destination	VARCHAR(100)	Student-declared destination
OutTime	TIMESTAMP	Null until scanned out
InTime	TIMESTAMP	Null until scanned in

ContactDirectories

Attribute	Type	Notes
ContactID	INT (PK, AUTO_INCREMENT)	
Category	VARCHAR(50)	e.g., Maintenance, HMC, Caretaker, Warden
Name	VARCHAR(100)	
PhoneNumber	VARCHAR(15)	
Details	TEXT	Additional notes or location info

TransportRoutes

Attribute	Type	Notes
RouteID	INT (PK, AUTO_INCREMENT)	
VehicleType	VARCHAR(30)	e.g., Bus, Ferry
Direction	VARCHAR(50)	e.g., From Campus

TransportSchedules

Attribute	Type	Notes
ScheduleID	INT (PK, AUTO_INCREMENT)	
RouteID	INT	FK -> TransportRoutes.RouteID
StopName	VARCHAR(100)	
DepartureTime	TIME	Static schedule time

CabShares

Attribute	Type	Notes
ShareID	INT (PK, AUTO_INCREMENT)	
HostID	INT	FK -> Users.UserID
Source	VARCHAR(100)	
Destination	VARCHAR(100)	
PickupDate	DATE	
PickupTime	TIME	

Attribute	Type	Notes
AvailableSeats	INT	Decremented on each approval
PhoneNumber	VARCHAR(15)	Host contact number
Notes	VARCHAR(255)	Optional
Status	VARCHAR(20)	Active, Full, Archived
CreatedAt	TIMESTAMP	Auto-set; used by expiry background job

CabRequests

Attribute	Type	Notes
RequestID	INT (PK, AUTO_INCREMENT)	
ShareID	INT	FK -> CabShares.ShareID
RequesterID	INT	FK -> Users.UserID
RequesterNote	VARCHAR(255)	Optional note from requester
Status	VARCHAR(20)	Pending, Approved, Rejected
RequestedAt	TIMESTAMP	

MedicalTimetables

Attribute	Type	Notes
DoctorID	INT (PK, AUTO_INCREMENT)	
DoctorName	VARCHAR(100)	
Specialization	VARCHAR(100)	
AvailableDays	VARCHAR(100)	e.g., Mon, Wed, Fri
AvailableTimeRange	VARCHAR(50)	e.g., 09:00–12:00

MarketplaceItems

Attribute	Type	Notes
ItemID	INT (PK, AUTO_INCREMENT)	
SellerID	INT	FK -> Users.UserID
AdType	VARCHAR(20)	Sell, Rent, Free

Attribute	Type	Notes
Title	VARCHAR(150)	
Description	TEXT	
AskingPrice	DECIMAL(10,2)	0.00 for free items
Status	VARCHAR(20)	Active, Sold, Closed, Hidden
CreatedAt	TIMESTAMP	Auto-set; used by expiry background job

ItemImages

Attribute	Type	Notes
ImageID	INT (PK, AUTO_INCREMENT)	
ItemID	INT	FK -> MarketplaceItems.ItemID
ImageURL	VARCHAR(300)	CDN/S3 path after compression
IsPrimary	BOOLEAN	Exactly one true per ItemID

Complaints

Attribute	Type	Notes
ComplaintID	INT (PK, AUTO_INCREMENT)	
UserID	INT	FK -> Users.UserID
PortalType	VARCHAR(20)	IPM, UPSP, Hostel
Category	VARCHAR(50)	e.g., Electrical, Plumbing, Academic
Description	TEXT	
ExternalIPMID	VARCHAR(50)	Populated only for IPM complaints
Status	VARCHAR(30)	Submitted, UnderReview, Assigned, Done
PhotoAttachmentURL	VARCHAR(300)	Optional
Timestamp	TIMESTAMP	

SSOPortalLinks

Attribute	Type	Notes
PortalID	INT (PK, AUTO_INCREMENT)	
PortalName	VARCHAR(100)	e.g., ERP, Fee Portal, Guest Room Booking
URL	VARCHAR(300)	SSO redirect or direct link
IsActive	BOOLEAN	Controls visibility in app
DisplayOrder	INT	Controls UI ordering

3. Functional Requirements

3.1 Transport

Bus/Ferry Schedules

–: The student should be able to view static schedules for bus/ferry stops. Each entry should show: vehicle type, stop name, direction, departure time.

–: Schedules are static and maintained via backend database updates; no real-time API is involved.

Peer-to-Peer Cab Sharing

–: A student ("Host") should be able to create a cab-share post with: source, destination, pickup date/time, available seat count, contact number, optional notes.

–: Other students should be able to browse active cab-share posts and submit a join request with an optional note.

–: The Host should be able to approve or reject incoming join requests from within the app.

–: On approval, the system should decrement AvailableSeats. When seats reach 0, the post should be automatically marked as Full.

–: A background job should automatically archive posts whose pickup date/time has elapsed.

–: Push notifications should fire: to the Host on a new join request; to the Requester on approval/rejection.

3.2 Campus Facilities

QR Code Gatelogs

–: The student should be able to generate a unique, encrypted QR code from the app for gate entry/exit. The student should be able to view their own historical gate log entries.

–: QR code should encode a student's identity, declared destination, and generation timestamp.

–: Each gate log entry should record: student identity, gate location, declared destination, OutTime, and InTime.

–: Gate scanning itself is handled externally by the device; the app only surfaces the QR and the log history to the student.

Guest Room Booking Portal

–: The app should display a clearly labelled redirect to the institute's external guest room booking portal.

–: No booking flow, availability check, or approval workflow is handled within this app. This link should be managed as an entry in SSOPortalLinks with a dedicated category flag.

Maintenance Contact Directory

–: The student should be able to browse a searchable, categorized directory of maintenance and facility contacts.

–: Contacts should be filterable by category (Electrical, Plumbing, Carpentry, Caretaker, Warden, HMC members, Hospital Contacts, E-rickshaw contacts, Food places' contacts).

–: The directory is read-only for the student; all updates are developer/admin-side database operations.

3.3 Welfare & Marketplace

Medical Section

–: The student should be able to view doctor timetables displaying: doctor name, specialization, available days, and available time range. Additionally, a daily calendar with availability of doctors should be floated and displayed.

–: The student should be able to view general health guidelines and emergency contact information.

–: All medical data is read-only in the app; it is populated and kept current via coordination with the medical services API on the backend.

Peer-to-Peer Student Marketplace

–: The student should be able to post a listing with: title, description, asking price, ad type (Sell/Rent/Free), and one or more images. The student needs to be identified with their Email-ID (phone number should be optional).

–: Uploaded images should be compressed before storage; one image should be designated as the primary display image.

–: The student should be able to manually update the status of their own listing (e.g., mark as Sold or Closed).

–: A background job should automatically set listings to Hidden once they exceed a defined age threshold, without any manual intervention.

–: The student should be able to browse all Active listings and filter by ad type.

Complaints Portal

–: The student should select a complaint channel before submitting: IPM (query to IPM database via external API) or UPSP/Hostel (form routed to internal Maintenance/Admin API).

–: All complaints are stored with: portal type, category, description, timestamp, optional photo attachment, and current status.

–: The student should be able to view a list of all their previously submitted complaints and their current statuses.

–: IPM: Student inputs an IPM ID -> the system fetches and displays the live status from external IPM portal API. If API is unreachable, show a static fallback status.

–: UPSP/Hostel: Student fills an online form -> it is routed to the internal Maintenance/Admin API -> the student receives push notifications as the status progresses.

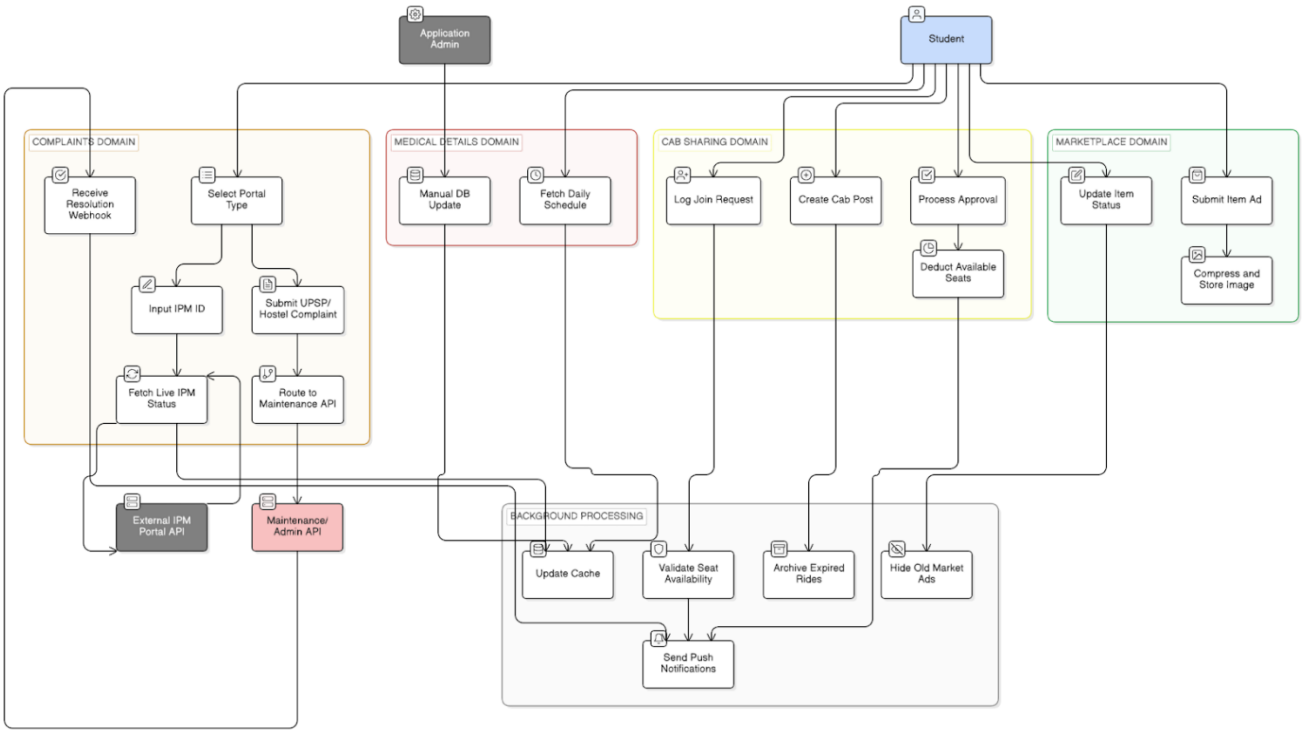
SSO Portal Links

–: The student should see the list of institute SSO portal quick links (e.g., https://online.iitg.ac.in/sso, https://academic.iitg.ac.in/sso).

–: Each link should open the corresponding portal via SSO-authenticated redirect where supported. The list is read-only for the student; it is managed via backend.

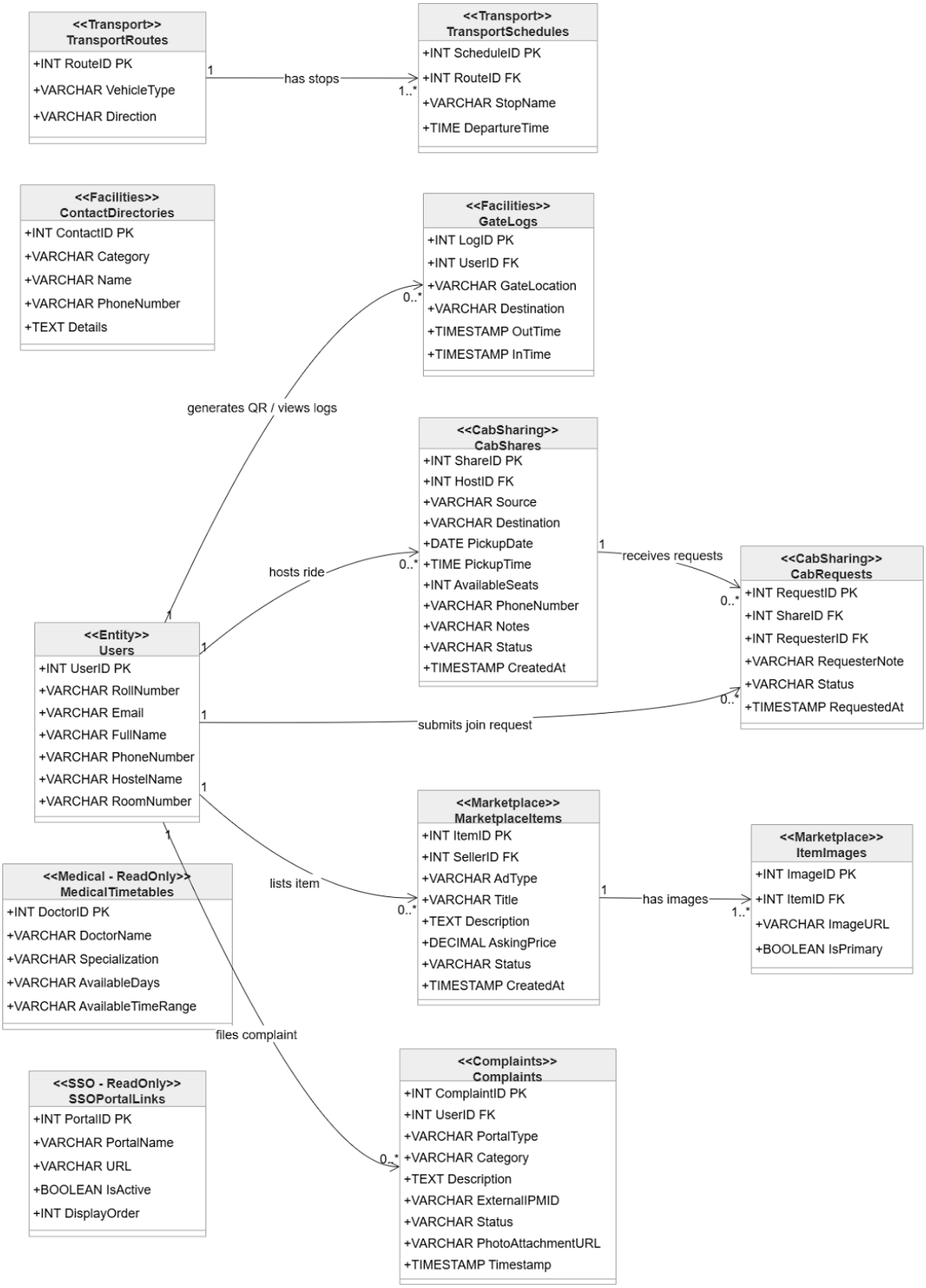
4. UML Diagrams

4.1 Use Case / Activity Diagram



eraser

4.2 Class Diagram


<<Transport>>
TransportRoutes
+INT RouteID PK
+VARCHAR VehicleType
+VARCHAR Direction
1
has stops
1..*
<<Transport>>
TransportSchedules
+INT ScheduleID PK
+INT RouteID FK
+VARCHAR StopName
+TIME DepartureTime
<<Facilities>>
ContactDirectories
+INT ContactID PK
+VARCHAR Category
+VARCHAR Name
+VARCHAR PhoneNumber
+TEXT Details
<<Facilities>>
GateLogs
+INT LogID PK
+INT UserID FK
+VARCHAR GateLocation
+VARCHAR Destination
+TIMESTAMP OutTime
+TIMESTAMP InTime
0..*
generates QR / views logs
1
<<CabSharing>>
CabShares
+INT ShareID PK
+INT HostID FK
+VARCHAR Source
+VARCHAR Destination
+DATE PickupDate
+TIME PickupTime
+INT AvailableSeats
+VARCHAR PhoneNumber
+VARCHAR Notes
+VARCHAR Status
+TIMESTAMP CreatedAt
hosts ride
0..*
1
receives requests
0..*
<<CabSharing>>
CabRequests
+INT RequestID PK
+INT ShareID FK
+INT RequesterID FK
+VARCHAR RequesterNote
+VARCHAR Status
+TIMESTAMP RequestedAt
<<Entity>>
Users
+INT UserID PK
+VARCHAR RollNumber
+VARCHAR Email
+VARCHAR FullName
+VARCHAR PhoneNumber
+VARCHAR HostelName
+VARCHAR RoomNumber
1
submits join request
0..*
1
lists item
0..*
<<Marketplace>>
MarketplaceItems
+INT ItemID PK
+INT SellerID FK
+VARCHAR AdType
+VARCHAR Title
+TEXT Description
+DECIMAL AskingPrice
+VARCHAR Status
+TIMESTAMP CreatedAt
1
has images
1..*
<<Marketplace>>
ItemImages
+INT ImageID PK
+INT ItemID FK
+VARCHAR ImageURL
+BOOLEAN IsPrimary
<<Medical - ReadOnly>>
MedicalTimetables
+INT DoctorID PK
+VARCHAR DoctorName
+VARCHAR Specialization
+VARCHAR AvailableDays
+VARCHAR AvailableTimeRange
1
files complaint
0..*
<<SSO - ReadOnly>>
SSOPortalLinks
+INT PortalID PK
+VARCHAR PortalName
+VARCHAR URL
+BOOLEAN IsActive
+INT DisplayOrder
<<Complaints>>
Complaints
+INT ComplaintID PK
+INT UserID FK
+VARCHAR PortalType
+VARCHAR Category
+TEXT Description
+VARCHAR ExternalIPMID
+VARCHAR Status
+VARCHAR PhotoAttachmentURL
+TIMESTAMP Timestamp

Module 3: Complaint Management

1. Stakeholder Identification

Stakeholder	Role	Key Interactions
Student	Primary User	Submits complaints with details and attachments, tracks complaint status, and provides feedback after resolution.
Admin/Warden	Decision Maker	Monitors complaints, assigns staff, manages workflow, and ensures timely resolution of issues.
Staff	Executor	Views assigned complaints, updates status, adds comments, and resolves issues reported by students.

2. Data Entities & Attributes

2.1 Complaint

Attribute	Type	Description
complaint_id	varchar	Unique identifier generated for each complaint to ensure traceability and management
student_id	varchar	References the student who submitted the complaint and links it to student records
category_id	varchar	Identifies the category of complaint such as electrical, plumbing, or infrastructure issues
status_id	int	Represents current state of complaint such as Pending, In Progress, or Resolved
priority_id	int	Indicates urgency level of complaint (Low, Medium, High) for prioritization
assigned_staff_id	varchar	Stores the staff member responsible for handling and resolving the complaint
title	varchar	Short summary describing the complaint issue for quick identification
description	text	Detailed explanation of the problem provided by the student including relevant context

Attribute	Type	Description
created_at	datetime	Timestamp when complaint was initially created and submitted into the system
updated_at	datetime	Timestamp of last modification or status update of the complaint

2.2 Student

Attribute	Type	Description
student_id	varchar	Unique identifier assigned to each student in the system
name	varchar	Full name of the student used for identification and communication
email	varchar	Official email address used for notifications and communication
hostel_block	varchar	Indicates the hostel block where the student resides
room_number	varchar	Specifies the exact room number allocated to the student
phone	varchar	Contact number used for urgent communication or updates
created_at	datetime	Timestamp indicating when the student record was created

2.3 Staff

Attribute	Type	Description
staff_id	varchar	Unique identifier assigned to each staff member in the system
name	varchar	Full name of staff responsible for handling complaints
email	varchar	Official email used for notifications and communication
role	varchar	Role of staff such as technician or supervisor
department	varchar	Department to which the staff member belongs

2.4 Complaint_Category

Attribute	Type	Description
category_id	varchar	Unique identifier for complaint category

Attribute	Type	Description
name	varchar	Name of category such as Electrical or Plumbing
description	varchar	Detailed explanation of category and type of issues it covers

2.5 Complaint_Status

Attribute	Type	Description
status_id	int	Unique identifier representing a specific status value
status_name	varchar	Name of status such as Pending, In Progress, or Resolved

2.6 Priority

Attribute	Type	Description
priority_id	int	Unique identifier for priority level
level	varchar	Indicates urgency such as Low, Medium, or High priority

2.7 Attachment

Attribute	Type	Description
attachment_id	varchar	Unique identifier for each uploaded file
complaint_id	varchar	Links attachment to corresponding complaint
file_name	varchar	Name of uploaded file or image
file_url	varchar	Storage path or URL where file is stored
uploaded_at	datetime	Timestamp indicating when file was uploaded

2.8 Notification

Attribute	Type	Description
notification_id	varchar	Unique identifier for each notification generated by system
complaint_id	varchar	Associates notification with a specific complaint
recipient_id	varchar	Identifies user receiving notification
recipient_type	varchar	Specifies whether recipient is student or staff

Attribute	Type	Description
message	text	Content of notification message sent to user
type	varchar	Type of notification such as update or assignment
is_read	boolean	Indicates whether notification has been read by user
sent_at	datetime	Timestamp when notification was sent

2.9 Comment

Attribute	Type	Description
comment_id	varchar	Unique identifier for each comment added to complaint
complaint_id	varchar	Links comment to specific complaint
user_id	varchar	Identifier of user who posted the comment
user_type	varchar	Specifies whether user is student or staff
message	text	Content of the comment providing updates or discussion
created_at	datetime	Timestamp when comment was created

2.10 Complaint_History

Attribute	Type	Description
history_id	varchar	Unique identifier for each history record
complaint_id	varchar	Links history record to complaint
old_status_id	int	Previous status before update
new_status_id	int	Updated status after change
changed_by	varchar	Identifier of user who made the change
changed_at	datetime	Timestamp when status change occurred

2.11 Feedback

Attribute	Type	Description
feedback_id	varchar	Unique identifier for feedback entry
complaint_id	varchar	Links feedback to resolved complaint
rating	int	Numeric rating provided by student based on resolution quality

Attribute	Type	Description
comments	text	Detailed feedback or suggestions provided by student
created_at	datetime	Timestamp when feedback was submitted

3. Functional Requirements

FR1 – Complaint Submission: The system shall allow students to submit complaints with category, title, description, priority, and optional attachments. A unique complaint ID shall be generated and stored with timestamps.

FR2 – Complaint Assignment: The system shall assign complaints to staff either manually by admin or automatically based on category or workload.

FR3 – Complaint Tracking: Students shall be able to view complaint status, history of updates, and detailed information of each complaint.

FR4 – Status Management: Staff shall update complaint status through stages such as Pending, In Progress, and Resolved.

FR5 – Complaint History Logging: Every status change shall be recorded in complaint history to maintain traceability.

FR6 – Notification System: System shall send notifications to users on submission, assignment, status updates, and resolution.

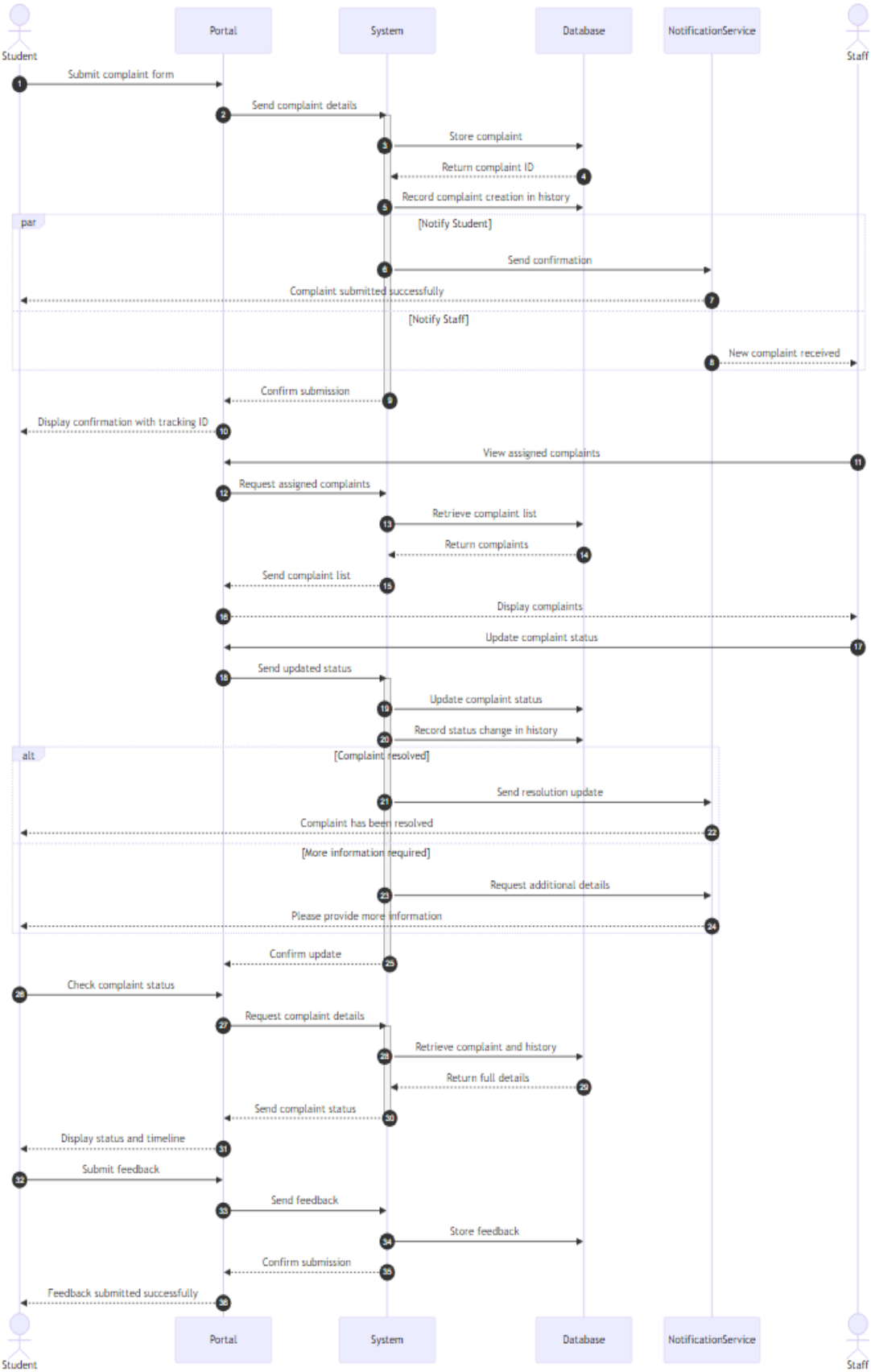
FR7 – Commenting System: Users shall be able to add comments to complaints for communication and clarification.

FR8 – Feedback System: Students shall provide feedback after complaint resolution including rating and comments.

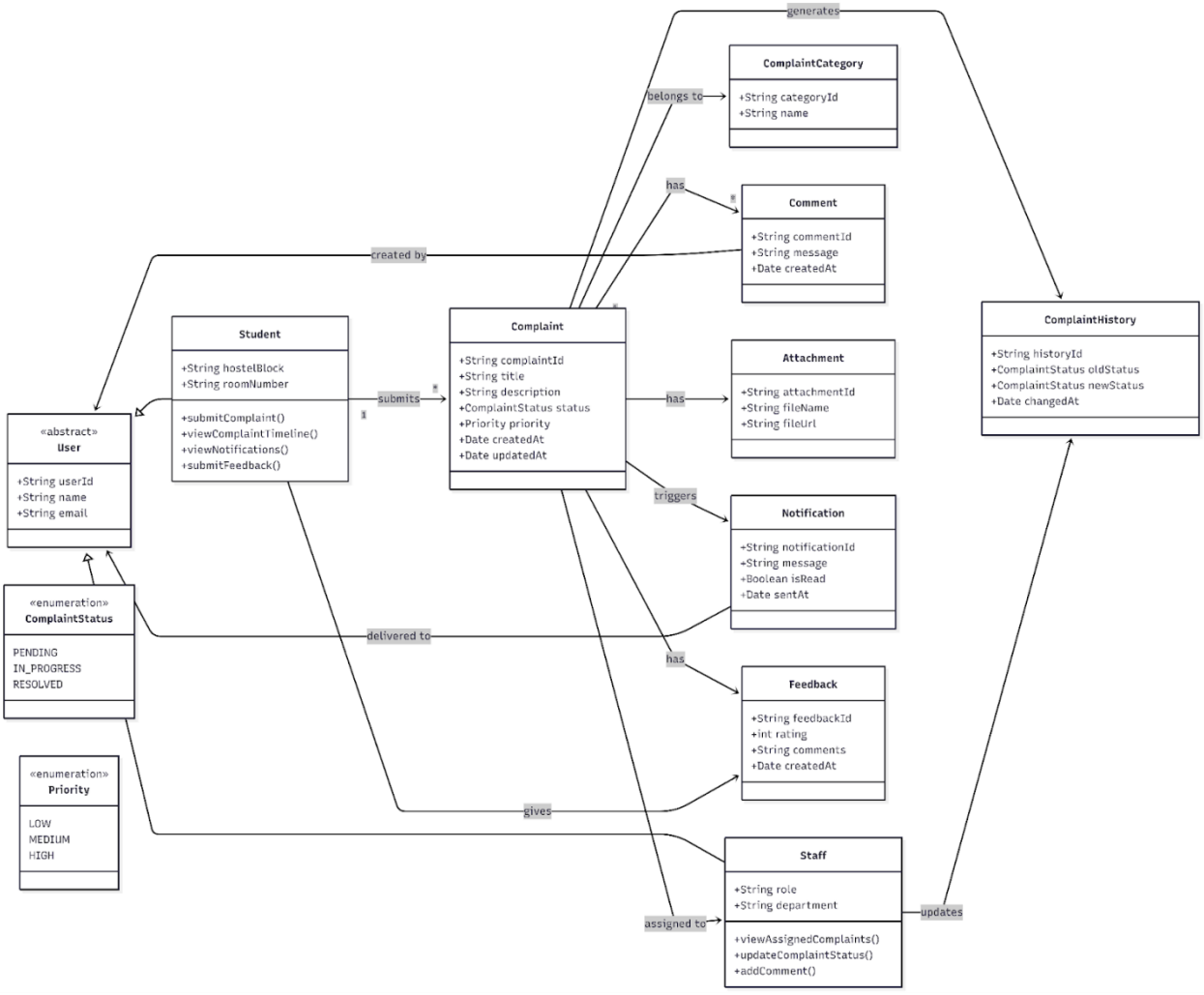
FR9 – Role-Based Access Control: System shall restrict actions based on user roles to ensure security and proper access.

4. UML Diagrams

4.1 Sequence Diagram


Student
Portal
System
Database
NotificationService
Staff
1 Submit complaint form
2 Send complaint details
3 Store complaint
4 Return complaint ID
5 Record complaint creation in history
par
[Notify Student]
6 Send confirmation
7 Complaint submitted successfully
[Notify Staff]
8 New complaint received
9 Confirm submission
10 Display confirmation with tracking ID
11 View assigned complaints
12 Request assigned complaints
13 Retrieve complaint list
14 Return complaints
15 Send complaint list
16 Display complaints
17 Update complaint status
18 Send updated status
19 Update complaint status
20 Record status change in history
alt
[Complaint resolved]
21 Send resolution update
22 Complaint has been resolved
[More information required]
23 Request additional details
24 Please provide more information
25 Confirm update
26 Check complaint status
27 Request complaint details
28 Retrieve complaint and history
29 Return full details
30 Send complaint status
31 Display status and timeline
32 Submit feedback
33 Send feedback
34 Store feedback
35 Confirm submission
36 Feedback submitted successfully
Student
Portal
System
Database
NotificationService
Staff

4.2 Class Diagram



Module 4: Authentication

1. Stakeholder Identification

Stakeholder	Role	Interactions
Student	Primary User	Logs into the system using institutional credentials; manages password resets and account recovery; updates basic profile information; manages active sessions across devices.
Professor	Primary User	Authenticates securely to access faculty-specific modules; manages password and multi-factor authentication (MFA) settings; updates faculty profile details.
Admin Staff	Administrator / Support	Manages user accounts, roles, and permissions (RBAC); handles bulk account creation; monitors authentication audit logs; assists with account lockouts and manual recovery.

2. Data Entities & Attributes

2.1 Student

Attribute	Type	Notes
login_id	int	Primary identifier
name	varchar	
roll_no	int	
outlook_mail_id	varchar	
department	varchar	
degree	varchar	
contact_no	varchar	
student_role	varchar	

2.2 Professor

Attribute	Type	Notes
login_id	int	Primary identifier

Attribute	Type	Notes
name	varchar	
outlook_mail_id	varchar	
department	varchar	
professor_post	varchar	
contact_no	varchar	
professor_role	varchar	

2.3 Admin_Staff

Attribute	Type	Notes
login_id	int	Primary identifier
name	varchar	
staff_designation	varchar	
outlook_mail_id	varchar	
contact_no	varchar	

2.4 Credentials

Attribute	Type	Notes
login_id	int	PK/FK
password_hash	varchar	Stored using bcrypt algorithm
last_password_changed	datetime	

2.5 Password_Reset

Attribute	Type	Notes
reset_id	int	PK
login_id	int	FK
token	varchar	Single-use reset token
created_at	datetime	
expiry_time	datetime	
is_used	Boolean	Marks whether token has been consumed

2.6 Session_Tracking

Attribute	Type	Notes
session_id	int	PK
login_id	int	FK
token	varchar	JWT token
device_info	varchar	
ip_address	varchar	
created_at	datetime	
expires_at	datetime	

3. Functional Requirements

FR1: The system must authenticate users by verifying their institutional credentials, such as their active Roll Number or official email, and password against the secure database.

FR2: System shall enforce Role-Based Access Control (RBAC) immediately upon login, dynamically restricting or granting access to specific portal modules based on whether the authenticated user is a Student, Admin, or Faculty.

FR3: User submits a "Forgot Password" request; system generates a time-sensitive, single-use reset link or OTP and dispatches it exclusively to the user's registered institutional email address.

FR4: The system must automatically lock a user's account for a predefined duration (e.g., 15 minutes) after five consecutive failed login attempts to prevent unauthorized access.

FR5: System shall track user activity and securely terminate the active session, returning the user to the login screen, after 30 minutes of continuous inactivity.

FR6: The system must maintain a secure audit log for every user account, recording timestamps, IP addresses, and the status (success/failure) of all login attempts and password changes.

FR7: Admin uploads a formatted CSV file for bulk user creation; the system parses the data, automatically generates initial passwords, and sends activation notifications to the new accounts.

FR8: System shall provide an interface for users to update their login passwords, enforcing standard complexity rules (minimum length, alphanumeric, special characters) before accepting the new password.

4. Hashing

For hashing of the password, we can use the “bcrypt” Algorithm, which is easily available in some libraries.

How it works:

1. **Salt Generation**: A random salt is generated and combined with the password.

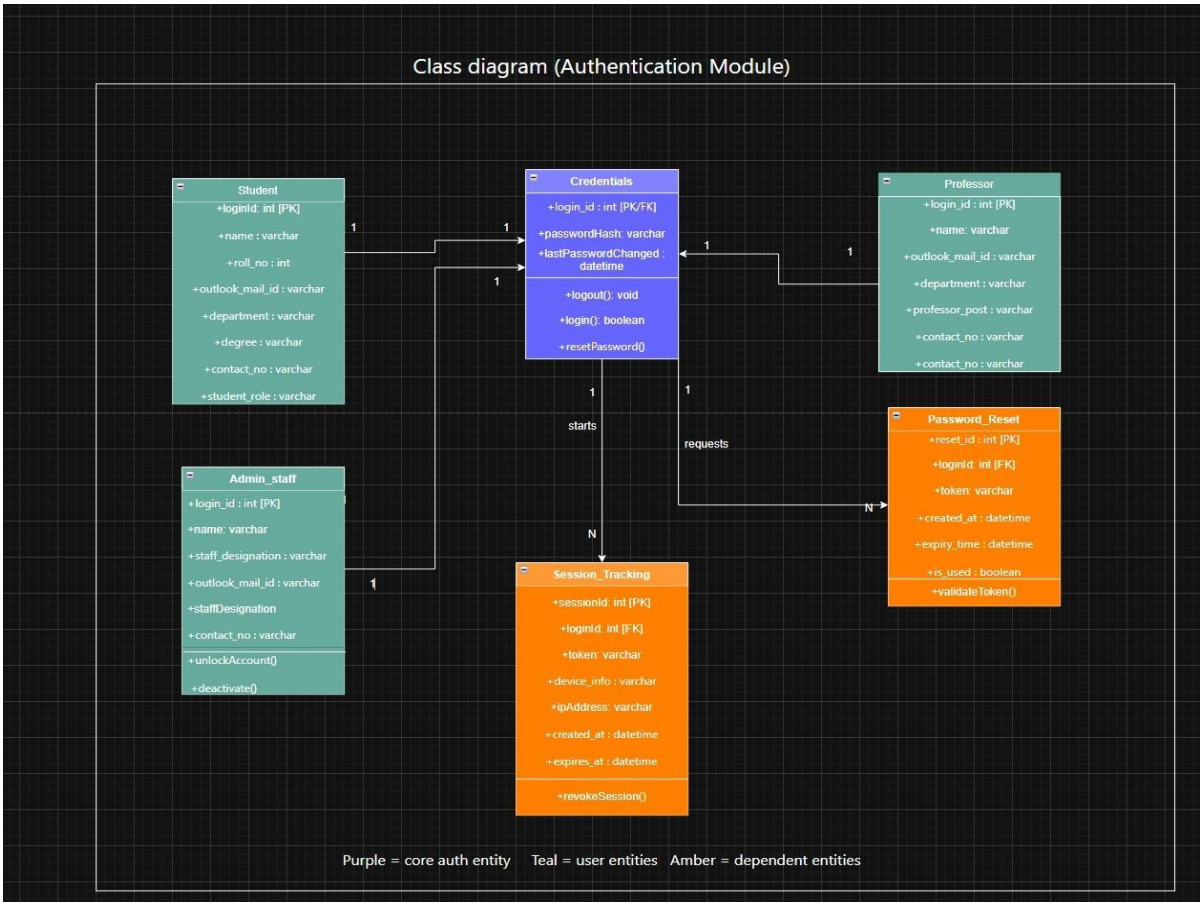
2. **Hashing Process:** The combined string is processed using the Blowfish cipher multiple times, determined by the "cost factor" (iterations).

3. **Storage**: The resulting hash (containing the salt and cost factor) is stored in the database.

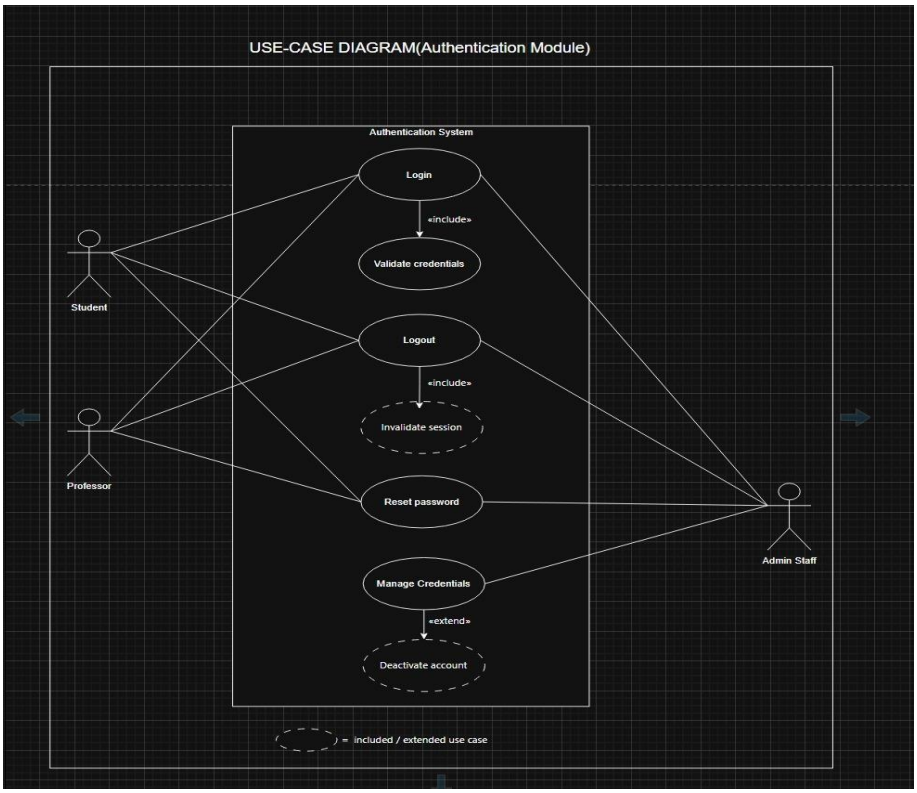
4. **Verification**: During login, bcrypt.compare() takes the plain text input and the stored hash to verify, making it easy to check without storing plain text passwords.

5. UML Diagrams

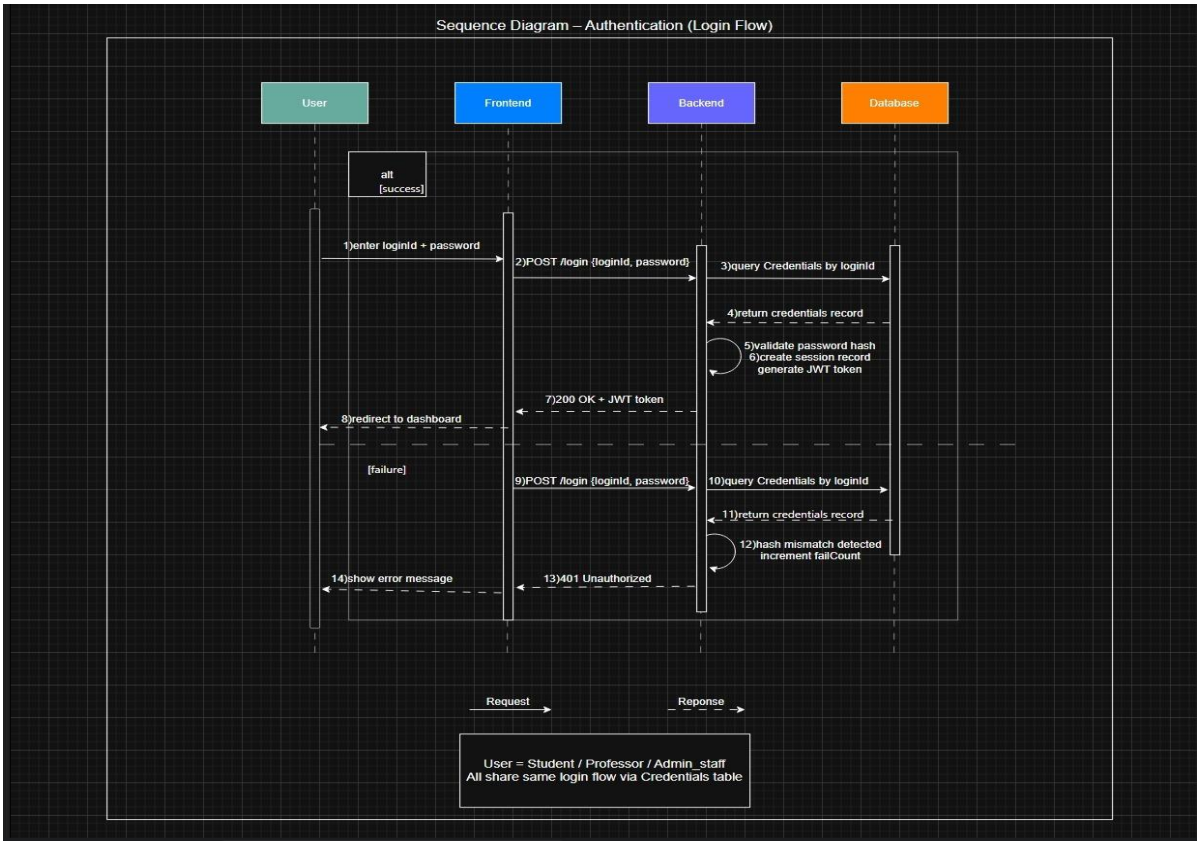
5.1 UML Class Diagram



5.2 UML Use Case Diagram



5.3 UML Sequence Diagram (Login Flow)



Module 5: Hostel Management System (HMS-SP)

1. Stakeholder Identification

Actor	Role	Technical Level
Student	Submits requests, feedback, service needs	Moderate – web portal user
Admin	Approves/rejects leave & transfer; audits	Moderate – trained admin staff
Mess Secretary	Approves rebates & subscriptions; billing	Low to Moderate
Maintenance Cell	Receives & resolves asset-linked complaints	Low – operational staff
Laundry Staff	Processes laundry & scans QR on completion	Low – operational staff

2. Data Entities & Attributes

Student (Core)

Attribute	Key	Notes
Roll_Number	PK	Primary identifier; hostel allocation pre-populated per semester
Hostel_ID	FK	Links to Hostel table
Room_Number		
Email		
Program		
Year		

Hostel (Core)

Attribute	Key	Notes
Hostel_ID	PK	
Name		

Attribute	Key	Notes
Total_Rooms		
Warden_Name		
Warden_Contact		

Asset (Maintenance)

Attribute	Key	Notes
Asset_ID	PK	
Hostel_ID	FK	
Asset_Type		
Location		
Condition		

Leave_Application (Core Admin)

Attribute	Key	Notes
Leave_Application_ID	PK	
Roll_Number	FK	
Dates		Start and end dates
Category		
Status		Pending/Approved/Rejected
Warden_Remarks		

Transfer_Request (Core Admin)

Attribute	Key	Notes
Request_ID	PK	
Roll_Number	FK	
Current_Hostel_ID	FK	
Target_Hostel_ID	FK	
Status		

Mess (Mess & OPI)

Attribute	Key	Notes
Mess_ID	PK	
Hostel_ID	FK	
Caterer_Name		
Capacity		
OPI_Rank		
OPI_Score		

Mess_Subscription (Mess & OPI)

Attribute	Key	Notes
Subscription_ID	PK	
Roll_Number	FK	
Mess_ID	FK	
Start_Date		
End_Date		
Is_Active		

Mess_Rebate (Mess & OPI)

Attribute	Key	Notes
Rebate_ID	PK	
Roll_Number	FK	
Leave_Application_ID	FK	
Dates		
Status		Pending/Approved

Mess_Feedback (Mess & OPI)

Attribute	Key	Notes
Feedback_ID	PK	
Roll_Number	FK	
Mess_ID	FK	
Meal_Rating		

Attribute	Key	Notes
Comments		

Mess_OPI_Monthly (Mess & OPI)

Attribute	Key	Notes
OPI_ID	PK	
Mess_ID	FK	
Month_Year		
Avg_Scores		
Final_OPI		
Rank		

Complaint (Maintenance)

Attribute	Key	Notes
Complaint_ID	PK	
Roll_Number	FK	
Asset_ID	FK	
Category		
Status		Open / In-Progress / Resolved
Timestamps		Created and resolved timestamps

Service_Request (Essential Services)

Attribute	Key	Notes
Request_ID	PK	
Roll_Number	FK	
Service_Type		Cleaning or Laundry
Status		
QR_Token		Unique token for laundry tracking
Scheduled_Date		

Policy_Window (Background)

Attribute	Key	Notes
Policy_ID	PK	
Policy_Type		
Start_Date		
End_Date		
Is_Active		

Notification (Background)

Attribute	Key	Notes
Notification_ID	PK	
Roll_Number	FK	
Actor_ID		
Event_Type		
Message		
Is_Read		

3. Functional Requirements

3.1 Authentication & Authorization

FR1 – Student Login: Accepts Roll Number + password. Validates against Student table. Issues JWT with role='student'. Returns authenticated dashboard or descriptive error.

FR2 – RBAC: Every API endpoint validates the JWT role claim. Returns HTTP 403 Forbidden for unauthorized access.

3.2 Core Admin Domain

FR3 – Apply for Leave: Input: Roll No., dates, category, destination, reason. Validates policy window, creates Leave_Application (status=Pending), notifies Admin.

FR4 – View Leave Status: Retrieves all Leave_Application records for the student with status (Pending/Approved/Rejected) and Warden_Remarks.

FR5 – Hostel Transfer: Input: target Hostel_ID, reason. Validates no active transfer exists. Creates Transfer_Request (status=Pending), notifies Admin.

3.3 Mess & OPI Domain

FR6 – Change Mess: Validates subscription policy window. Deactivates current Mess_Subscription and creates new one for target mess. Notifies Mess Secretary.

FR7 – Mess Rebate: Validates linked approved Leave_Application_ID; checks no duplicate claim. Creates Mess_Rebate (status=Pending) for Secretary approval.

FR8 – Mess Feedback: Records per-meal ratings and comments. Subscriber feedback carries higher OPI weight. Secretary can view aggregated ratings.

FR9 – OPI Calculation: Background system computes monthly OPI from subscriber meal ratings, SMC scores, and hygiene audits. Updates Mess_OPI_Monthly and rank.

3.4 Maintenance Domain

FR10 – File Complaint: Student selects asset from hostel directory. Creates Complaint linked to Asset_ID (status=Open). Maintenance Cell notified. Cell updates status to In-Progress → Resolved with timestamps.

FR11 – View Complaint Status: Student views their complaint history with live status and optional resolution notes from Maintenance Cell.

3.5 Campus Shops Domain

FR12 – Canteen Feedback: Ratings for overall quality, portion size, food quality, and pricing. Admin compliance dashboard updated.

FR13 – Juice/Stationery: Respective feedback records (fruit freshness, hygiene, stock availability, pricing) inserted. Admin view for audit.

3.6 Essential Services Domain

FR14 – Room Cleaning: Validates free cleaning quota. Creates Service_Request (type=Cleaning). Rejects if quota exhausted; displays current balance.

FR15 – Laundry QR: Creates Service_Request (type=Laundry) + unique QR_Token. Staff scans QR upon completion → status=Ready. Push notification sent to student.

3.7 Background Processing System

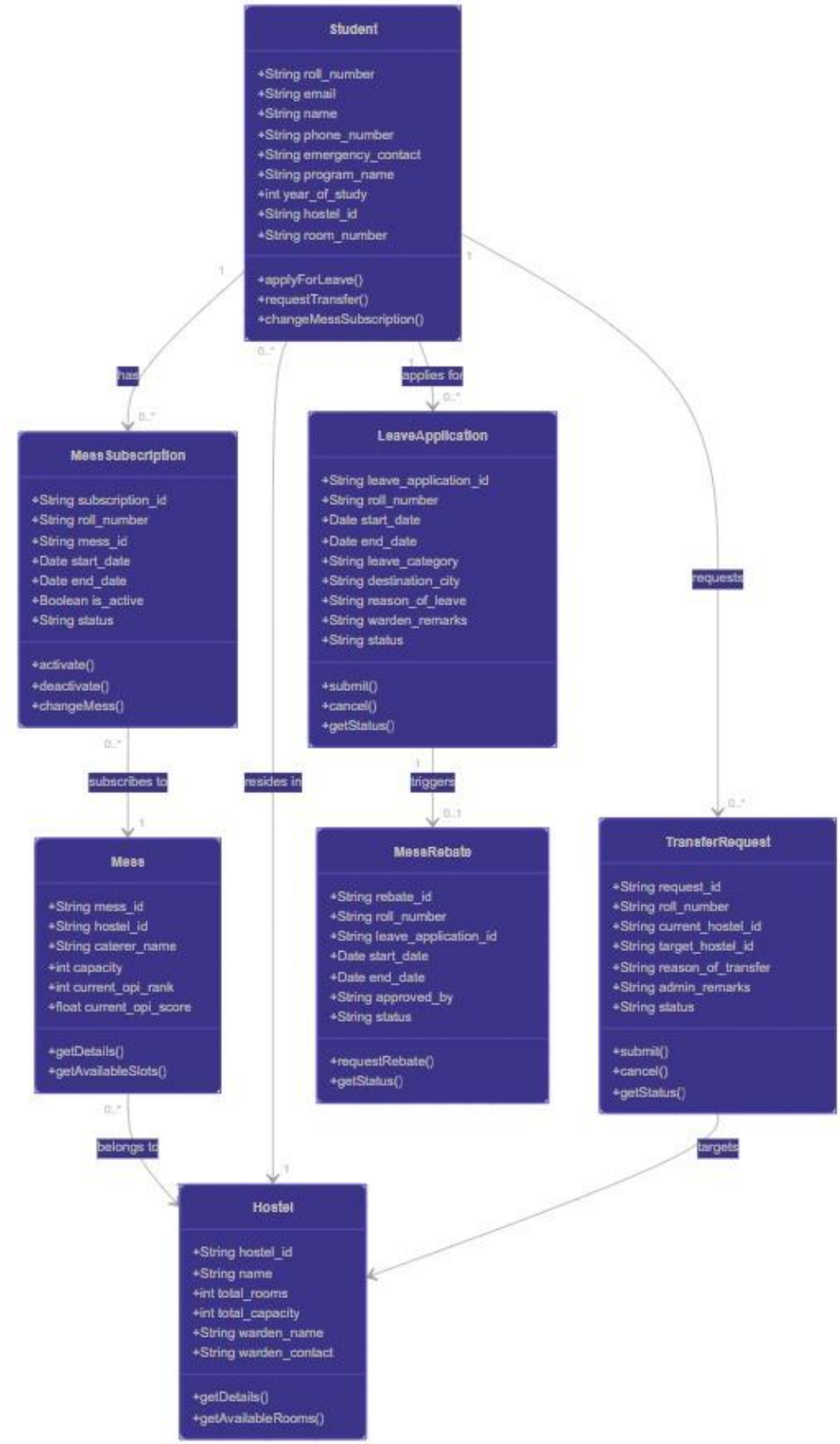
FR16 – Policy Validation: Maintains configurable windows for leave, subscriptions, and rebates. Auto-rejects out-of-window requests with next-period message.

FR17 – Notifications: Dispatches alerts on: leave submission (→Admin), transfer (→Admin), complaint (→Maintenance), rebate (→Secretary), laundry ready (→Student).

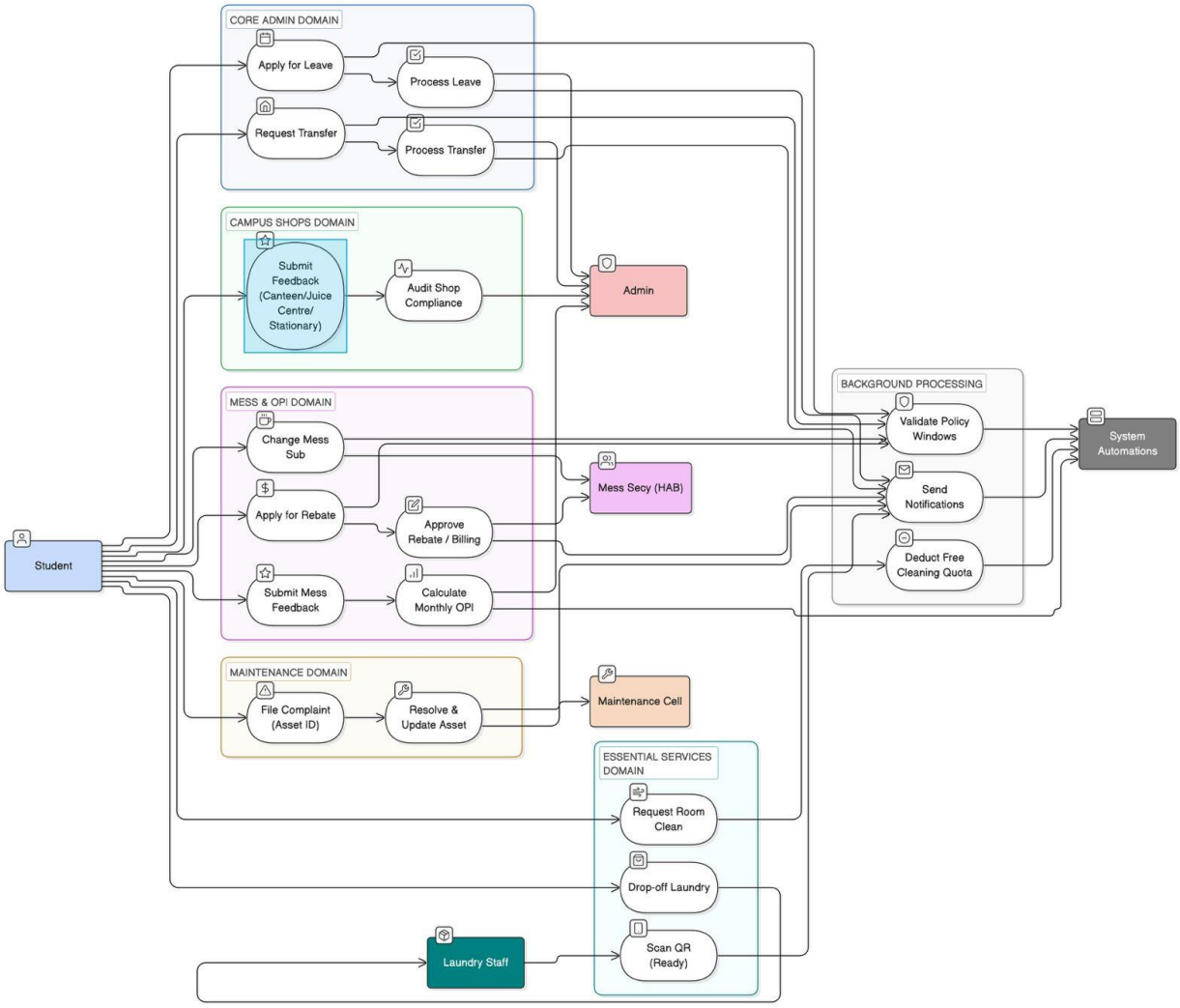
FR18 – Quota Deduction: Auto-decrements free cleaning quota on service confirmation. Flags account at zero; replenishes at start of next academic period.

4. UML Diagrams

4.1 Class Diagram


Student
+String roll_number
+String email
+String name
+String phone_number
+String emergency_contact
+String program_name
+int year_of_study
+String hostel_id
+String room_number
+applyForLeave()
+requestTransfer()
+changeMessSubscription()
1
1
0..*
has
applies for
requests
MessSubscription
+String subscription_id
+String roll_number
+String mess_id
+Date start_date
+Date end_date
+Boolean is_active
+String status
+activate()
+deactivate()
+changeMess()
LeaveApplication
+String leave_application_id
+String roll_number
+Date start_date
+Date end_date
+String leave_category
+String destination_city
+String reason_of_leave
+String warden_remarks
+String status
+submit()
+cancel()
+getStatus()
subscribes to
resides in
triggers
Mess
+String mess_id
+String hostel_id
+String caterer_name
+int capacity
+int current_opi_rank
+float current_opi_score
+getDetails()
+getAvailableSlots()
MessRebate
+String rebate_id
+String roll_number
+String leave_application_id
+Date start_date
+Date end_date
+String approved_by
+String status
+requestRebate()
+getStatus()
TransferRequest
+String request_id
+String roll_number
+String current_hostel_id
+String target_hostel_id
+String reason_of_transfer
+String admin_remarks
+String status
+submit()
+cancel()
+getStatus()
belongs to
targets
Hostel
+String hostel_id
+String name
+int total_rooms
+int total_capacity
+String warden_name
+String warden_contact
+getDetails()
+getAvailableRooms()

4.2 Sequence Diagrams



eraser

Module 6: Campus Management System

1. Stakeholder Identification

Stakeholder	Role	Key Interactions
Student	Primary User	Views profile, timetable & academic records; submits complaints and update requests; registers for events; uses marketplace; generates QR gate pass; manages mess subscription; requests certificates and ID card.
Board Exec	Stakeholder	Oversees event and gymkhana activities on the platform.

2. Data Entries and Attributes

Event

Attribute	Key	Notes
eventId	String (PK)	Unique identifier for the event
title	String	Title/name of the event
eventDate	Date	Date and time of the event
category	String	Category (Cultural, Technical, Sports, Academic, etc.)
eventId	String (PK)	Unique identifier for the event

Event Registration

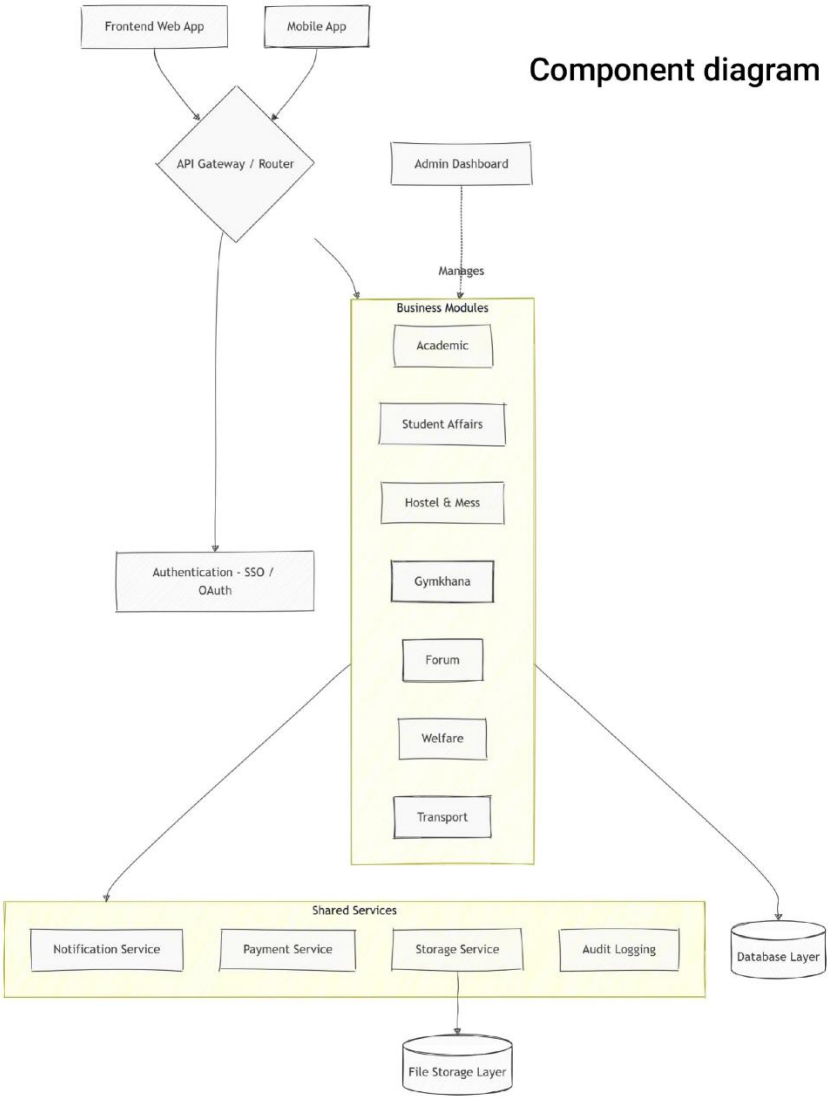
Attribute	Data Type	Description
registrationId	String (PK)	Unique identifier for the event registration
eventId	String (FK)	References the Event entity
userId	String (FK)	References the registering student
status	String	Registration status (Registered, Waitlisted, Cancelled)
Attribute	**Data Type**	**Description**

3. Functional Requirements

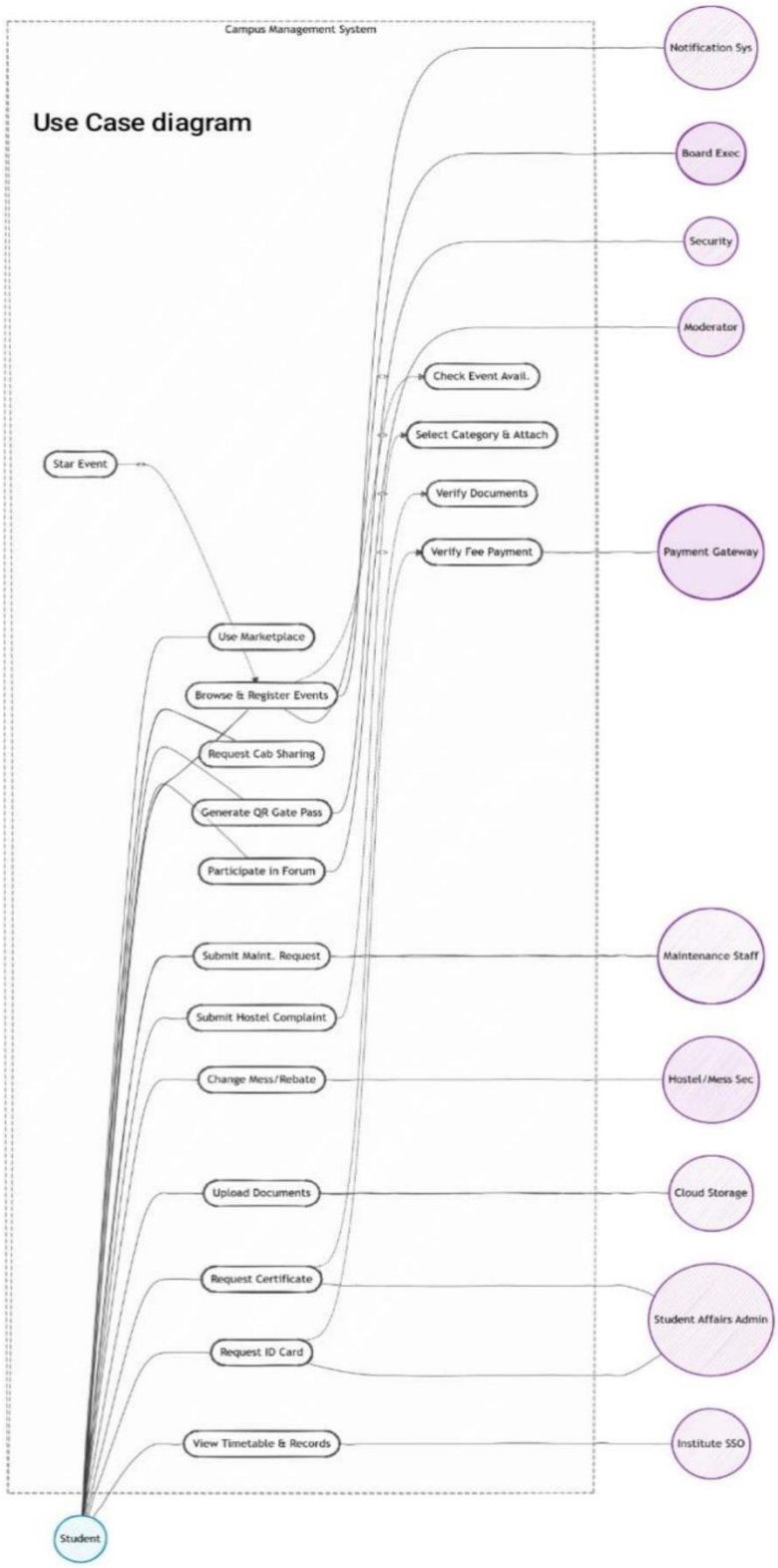
- **FR1:** Board Exec must be able to create and publish events with title, date, and category.
- **FR2:** Students must be able to browse active events, check availability, and register.
- **FR3:** The system must verify fee payment through the Payment Gateway before confirming paid event registrations.
- **FR4:** Students must be able to star/bookmark events for quick access.
- **FR5:** The system must send registration confirmation notifications to students.

4. UML Diagrams

4.1 Component Diagram



4.2 Use Case Diagram



4.3 Class Diagram